Manil Bastola
Stephanie Mertz
CSE 566S
Homework 1

All experiments in this paper were performed on a PC laptop with 64 bit-Ubuntu 16.04 OS. Other specs include:

Hardware:

HDD : 0.5TB SSD
processors : 3
vendor_id : GenuineIntel
cpu family : 6
model : 37
model name : Intel(R) Core(TM) i5 CPU M 450 @ 2.40GHz
stepping : 5
microcode : 0x2
cpu MHz : 1199.000
cache size : 3072 KB
physical id : 0
siblings : 4
cpu cores : 2

Memory:

	total	used	free	shared	buffers	cached
Mem:	3906500	3105516	800984	92340	52168	710244
-/+ buffers/cache:		2343104	1563396			
Swap:	7999484	31268	7968216			

1.) Compare and contrast the profilers we discussed in class (gprof, oprofile, perf). What are the benefits and disadvantages of each one? Do they agree with each other? If not, why? Feel free to swap out one or two of the profilers for other profiling tools that you know of. Which profilers are better in terms of accuracy? How does the overhead of each profiler affect its results? Which profiler do you prefer?

The profilers work by statistically sampling the program at a certain frequency to determine where the program currently is. Gprof does not know how much time the program is in libraries or kernel events so sometimes the results can look a bit skewed like with the in-class example of where multiply was taking so long. Oprofile on the other hand can see how long it is taking in the libraries and kernel events leading to a potentially richer analysis. This ability is because Oprofile is also triggered on hardware events. Perf is highly integrated with linux and with sudo privileges can extract low level kernel calls that both gprof and oprofile miss.

Gprof is simple tool and is useful for ballparking the time spent on certain functions. Oprofile does not show the time spent but shows the number of cpu clock cycle (samples) spent on different functions. It can additionally show resource related info such as cache hit/misses, samples spent on linked (non native) functions,etc. Perf can provide detailed kernel level information such as context switches, task_ticks, ioread etc.

We conducted a simple experiment to observe the similarities among the profilers. hw1.c was observed using three profilers. We set N=1000 and performed 1000 repetitions of vm_multiply to reduce timing variances. Also the optimization flag was set to -O0 since grof's output was erroneous for -O2 compilation flag. Each experiment was repeated 6 times and averages were calculated with uncertainties given by the t-distribution. The following are the results:

	initialize()	**vm_multiply()**
Gprof	0.011 ± 0.025 s	25.53 ± 0.31 s
Oprofile	0.028 ± 0.008 %	99.6 ± 0.17 %
Perf	0.0275 ± 0.13 %	99.39 ± 0.16 %

One can observe that gprof has high uncertainties for shorter programs. For example, initialize() took about 0.011s on average but the uncertainties in that value is about 0.025 seconds, which is around 227% of uncertainty! This shows lack of precision on part of gprof. However it has acceptable uncertainties for longer programs, for example vm_mulitply(). The other profilers did not show time spent on each functions directly so we show the percentage time spent on each function instead. Observing such results, I think we would use grof for simple timing analysis but for more complex understanding of behind the scenes, we would go with perf for its detailed (low kernel level
) report.

2.) What can we learn from simple end-to-end timing, or manual insertion of timing hooks? Compare the use of timing hooks with the profilers you examined in question 1.

Simple end-to-end timing using a profiler such as gprof can be a really great way to get a rough idea on where the code is spending the bulk of its time when you start to try to optimize a process. It is possible to get as granular of a view of the code performance, especially from a hardware perspective such as cache-misses, with timing hooks. This is a great way to begin to identify and target potential inefficiencies. End to end timing can be useful if you'd like to have some rough measurements of kernel-level events. Another advantage is that timing hooks may not be easy to insert into some events. For example, we may call library functions that are linked to our code. It can be very difficult and impossible in some cases to insert hooks in those library

functions. Meanwhile, the profilers such as oprofile and perf can import the namespaces being used and add their own hooks to analyse the linked code.

However, profilers tend to bring in measurement errors i.e. the more they measure the state of the running code, the more it interrupts the code and more errors related to timing increases. Hence once we find the time and resource intensive portion of the code, we can use manual insertions to find a better estimate of the time spent on that portion of the code.

We conducted a simple experiment to analyse hw1.c using manual timing hooks. Same as 1), we set N=1000 and performed 1000 repetitions of vm_multiply. The optimization flag was set to -O0. Each experiment was repeated 6 times and averages were calculated with uncertainties given by the t-distribution. The following is the result:

	initialize()	**vm_multiply()**
Manual Timing Hooks	0.019 ± 0.007 s	25.27 ± 0.68 s

One can observe similar average timing result for both functions with the result of gprof. However the percentage uncertainty in initialize (~ 37%) using manual hook is far smaller than the percentage uncertainty using gprof (~ 227%). The large uncertainty in grpf was due to the higher effect of sampling time for a shorter program. This shows that for small programs, manual function hooks are better than sampling the function with grpof. However for more details analysis oprofile or perf is recommended.

4.) What can we learn about optimization from the compiler (e.g. -O)?

Compilers can transform the code in attempt to optimize execution time as a tradeoff for compile time and potentially memory. We may choose to let compilers (for example, gcc) to optimize our code by adding -O flags. In our experiment, we tried compiling hw1.c using four different -O flags. The size of N was set to 1000 and the program performed 1000 vector matrix multiplications. The compilation time was calculated using a bash script (see Appendix). The results are shown below:

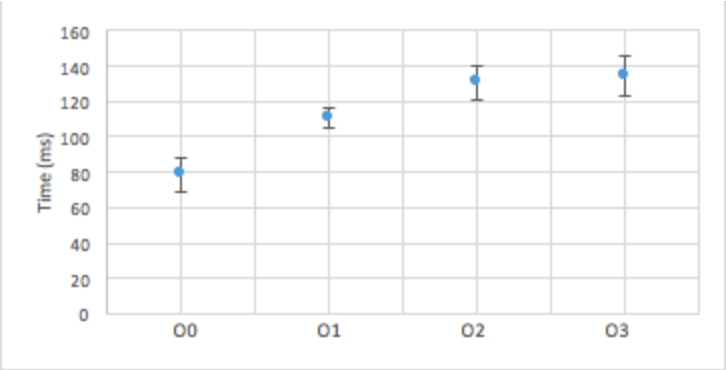


Fig: Compilation time for Different Optimization flags

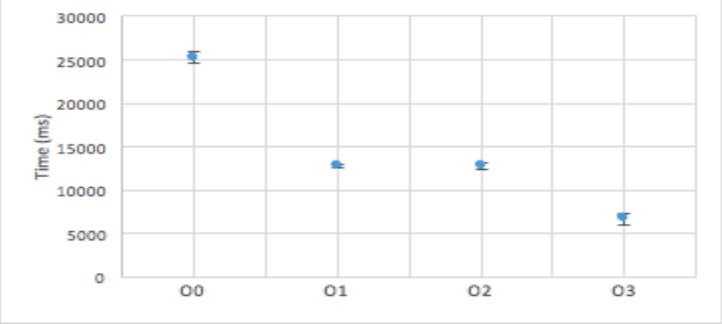


Fig: Running time of VM multiply for Different Optimization flags

One can observe that moving from -O0 to -O3 caused about 75% increase in the compilation time while about 75% decrease in the running time of the function vm_multiply(). The effect however is not linear. For example, -O0 to -O1 cause about 50% decrease in the running time while 37% increment in compile time. Meanwhile, though compilation time increased for -O1 to -O2, there was no significant improvement in the running time. FInally, with negligible increment in compile time (~ 3%) we achieved about 47% improvement in running time moving from -O2 to -O3 optimization.

5.) How about pre- versus post- multiplication (i.e. r = vM or r = Mv)?
The results of pre- versus post- multiplication are different. This is expected since pre-multiplication does the inner product while post-multiplication does the outer product of the vector v with each columns of matrix M. We compare our implementation (see Appendix) with Matlab for mathematical accuracy:

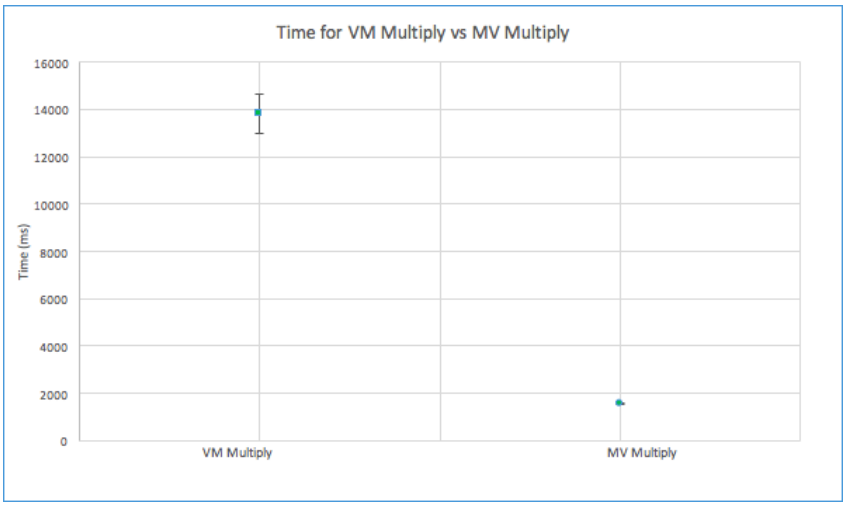
output:

```
manil@manil-Q430-Q530:~/cse566/hw1$ ./hw1
vector:
 0.8402 0.9116 0.2778
matrix:
row 0:
 0.3944 0.7831 0.7984
row 1:
 0.1976 0.3352 0.7682
row 2:
 0.5540 0.4774 0.6289
result:
vM multiply: 0.6653 1.0962 1.5459
Mv' multiply: 1.2671 0.6850 1.0753
```



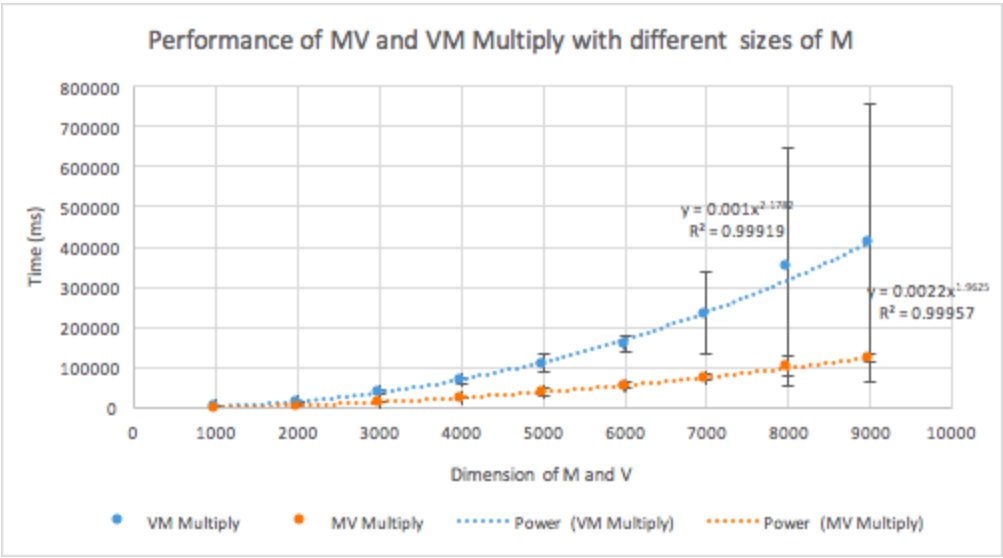
As implemented (see Appendix), we hypothesized that the results of the two functions differ also in the time needed to compute them. We think post-multiplication is faster than pre-multiplication. This is because of spatial locality in cache. When an element of M is accessed, the entire row (or a portion of the row if the row is very large) is brought to the cache. Thus, the row order traversal is faster because of large cache hit to miss ratio. However, when calculations are done in column order i.e. we access different column in the innermost loops iteration, it forces cache to flush the previous row after each iteration. This significantly increases cache misses, increasing the time it takes to perform the multiplication.

An experiment was setup with -O2 flag for compilation. The size N was set to 1024 and the program performed 1000 vector matrix pre and post multiplications. The result is shown below. As hypothesized, post-multiplication came out to be around 88% faster than pre-multiplication. Finally, the error bars in VM multiply are significantly larger than MV multiply. We think that since the running time of VM multiply is about 88% longer than MV multiply, there are more ways (OS related) to induce errors, context switches, background programs, etc. for example.



6.) What is the performance dependence on the size of the matrix or vector?

We hypothesize that the size of multiplying a vector of length N and a matrix of size NxN is bounded in O(N^2). This to be the case because for our implementation, number of operations would be proportionate to the number of columns times the number of rows due of two nested for loops.
In a new experiment, we changed N from 1000 to 9000 with increments of 1000. We observed both pre and post multiplication to see effects of spatial locality on increasing matrix size. The timing was measured using manual timing hooks as mentioned in 2 (or see Appendix). Like other experiments the number of repetitions of matrix multiplication was 1000. The compiler had O2 flag for optimizations. Finally, each experiment was conducted 5 times for statistical soundness. Our results were as follows:



Using excel we were able to calculate fit equation of y = 0.001x^2.1782 for vM multiply with an R^2 of 0.99919 and y = 0.0022x^1.9625 for Mv' multiply with an R^2 of 0.99957. The slight deviances from a*x^2 is due to practical reasons such as optimizations (-O2), machine states and cache policies. For example, due to spatial locality, Mv' multiply performs better than vM multiply for increasing N as expected. Finally, like 5) the error bars in VM multiply are significantly larger than MV multiply. Again, we argue that since the running time of VM multiply is significantly longer than MV multiply, there are more ways (OS related) to induce errors: context switches, background programs etc.

APPENDIX

file: make.sh

```
#computes compile time of hw1.c
!/bin/bash

time1=$(date +%s%3N)
make norm
time2=$(date +%s%3N)
echo $(( $time2 - $time1 ))
```

file: hw1.c

```
/* hw1.c
 *
 * Simple vector-matrix multiply code.
 *
 */

#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>


#define N 1024                  // length of vector, length and width of matrix
#define SEED 1                  // random number seed
#define REPS 1000    // repetitions
#define OUT   // output flag

double vector[N];
double matrix[N][N];
double result[N];

void initialize();
```

```
void initialize2(int n);
void vm_multiply();
void mv_multiply();
void output();

float timedifference_msec(struct timeval t0, struct timeval t1)
{
  return (t1.tv_sec - t0.tv_sec) * 1000.0f + (t1.tv_usec - t0.tv_usec)/ 1000.0f;
}


int main() {
  int i,j;
  struct timeval begin1, end1, begin2, end2;
  for (j=1000; j < N; j+=(N/10)){
    //gettimeofday(&begin1, NULL);
    initialize2(j);
    //gettimeofday(&end1, NULL);
    //printf("initialize for N= %i used %f milliseconds to complete theexecution\n", j ,
timedifference_msec(begin1, end1));
    //gettimeofday(&begin2, NULL);
    //clock_gettime(CLOCK_MONOTONIC_RAW, &begin1);
    //begin1 = time(NULL);
    for (i=0; i<REPS; i++) {
      vm_multiply(j);
    }
    //printf("Done");

    //end1 = time(NULL);
    //clock_gettime(CLOCK_MONOTONIC_RAW, &end1);
    //gettimeofday(&end2, NULL);
    //printf("VM multuply for N= %i used %f milliseconds to complete theexecution\n", j ,
timedifference_msec(begin2, end2));
    /*
    //output();
    gettimeofday(&begin2, NULL);
    for (i=0; i<REPS; i++) {
      mv_multiply(j);
    }
    gettimeofday(&end2, NULL);
    printf("%i %f %f\n", j , timedifference_msec(begin1, end1), timedifference_msec(begin2,
end2));
  //output();*/
```

```
  }
  return 0;
}

void initialize() {
  int i,j;
  srand(SEED);
  for (i=0; i<N; i++) {
    vector[i] = (double)rand()/RAND_MAX;
    result[i] = 0.0;
    for (j=0; j<N; j++) {
      matrix[i][j] = (double)rand()/RAND_MAX;
    }
  }
}

void initialize2(int n) {
  int i,j;
  srand(SEED);
  for (i=0; i<n; i++) {
    vector[i] = (double)rand()/RAND_MAX;
    result[i] = 0.0;
    for (j=0; j<n; j++) {
      matrix[i][j] = (double)rand()/RAND_MAX;
    }
  }
}

void vm_multiply(int n) {
  int i,j;
  for (j=0; j<n; j++) {
    result[j] = 0.0;
    for (i=0; i<n; i++) {
      result[j] += vector[i]*matrix[i][j];
    }
  }
}

void mv_multiply(int n) {
  int i,j;
  for (j=0; j<n; j++) {
    result[j] = 0.0;
    for (i=0; i<n; i++) {
```

```
      result[j] += matrix[j][i]*vector[i];
    }
  }
}


void output() {
#ifdef OUT
  int i,j;
  printf("vector:");
  for (i=0; i<N; i++) {
    if (i%10 == 0)
      printf("\n");
    printf(" %6.4f",vector[i]);
  }
  printf("\n");
  printf("matrix:\n");
  for (i=0; i<N; i++) {
    printf("row %d:",i);
    for (j=0; j<N; j++) {
      if (j%10 == 0)
            printf("\n");
      printf(" %6.4f",matrix[i][j]);
    }
    printf("\n");
  }
  printf("result:");
  for (i=0; i<N; i++) {
    if (i%10 == 0)
      printf("\n");
    printf(" %6.4f",result[i]);
  }
  printf("\n");
#else
  return;
#endif
}
```